LoRA: Low-Rank Adaptation of Large Language Models

Xu Zhang Dennis Chen Selina Lin

1. Introduction and Motivation

Large pretrained language models are powerful but expensive to adapt because standard full fine-tuning updates every model parameter for each downstream task. As model sizes grow, this becomes increasingly inefficient in both computation and storage: each task may require a separate copy of a fully fine-tuned model. The LoRA paper addresses this problem by proposing Low-Rank Adaptation, a parameter-efficient fine-tuning method that freezes the pretrained model weights and trains only small low-rank update matrices inserted into selected Transformer layers.

Our project re-implements and evaluates LoRA using RoBERTa-base [6] on GLUE classification tasks, focusing on SST-2 and QNLI [1]. We study whether LoRA can achieve strong validation performance while training far fewer parameters than full fine-tuning. In addition to reproducing the core LoRA setup, we extend the experiment by varying the LoRA rank r to examine how much model capacity is actually needed for effective adaptation. This lets us test one of the paper's central claim that low-rank updates are sufficient for adapting large pretrained models to downstream tasks.

2. Methodology

We re-implemented LoRA using RoBERTa-base as the pretrained backbone and evaluated it on two GLUE [5] benchmark tasks: SST-2 [3] and QNLI [4]. SST-2 is a binary sentiment classification task, while QNLI asks whether a sentence contains the answer to a given question. For both tasks, we used Hugging Face Transformers, Datasets, Trainer, and PEFT [7] to load the model, preprocess the data, train LoRA adapters, and evaluate validation accuracy.

LoRA freezes the original pretrained weight matrix W_0 and learns a low-rank update instead of directly fine-tuning all parameters. The adapted forward pass can be written as:

$$h = W_0 x + BAx$$

where A and B are trainable low-rank matrices with rank r, and r is much smaller than the original matrix dimensions. Following the LoRA paper, we applied LoRA to the query and value projection matrices in RoBERTa's attention layers. The pretrained backbone remained frozen, while only the LoRA parameters and classification head were trained. We used validation accuracy as the primary metric and compared LoRA against a baseline RoBERTa model and full fine-tuning behavior where applicable.

3. Experimental Setup

We conducted our experiments in Google Colab using GPU acceleration. The implementation used PyTorch, Hugging Face Transformers, Datasets, Trainer, and PEFT. For both SST-2 and QNLI, we loaded the GLUE dataset [5] through Hugging Face Datasets and tokenized each example with the RoBERTa tokenizer using truncation and padding to a maximum sequence length of 128. SST-2 examples were tokenized as single sentences, while QNLI examples were tokenized as question-sentence pairs.

For the LoRA configuration, we applied adapters to the query and value projection matrices in RoBERTa's self-attention layers. Unless otherwise noted, we used dropout 0.1, batch size 32, weight decay 0.01, and mixed precision training. For the rank experiment, we evaluated ranks $r \in \{1, 2, 4, 8, 16, 32\}$ and used $\alpha = 2r$ for the LoRA scaling factor. The QNLI rank sweep used a learning rate of 2×10^{-4} and trained for 5 epochs, while the SST-2 experiments used a learning rate of 5×10^{-4}. After training, we evaluated each model on the validation split and reported validation accuracy.

4. Results

Our experiments show that LoRA substantially improves over the unfine-tuned RoBERTa baseline on both SST-2 and QNLI. On SST-2, the baseline validation accuracy was approximately 0.509, while LoRA reached about 0.674 with rank 1 and improved to roughly 0.724–0.728 for higher ranks. The best SST-2 result occurred around rank 16, after which performance stayed nearly flat. This supports the LoRA paper's claim that a small low-rank update can provide most of the adaptation benefit 1.

On QNLI, the baseline validation accuracy was approximately 0.505. LoRA again improved performance across all tested ranks, reaching about 0.718 at rank 1 and increasing to about 0.739 at rank 32. Unlike SST-2, QNLI showed a smoother upward trend as rank increased, but the gains became relatively small after the lower ranks. In both tasks, validation accuracy rose quickly from very small ranks and then stabilized, suggesting diminishing returns from increasing LoRA rank beyond a modest value 2.

We also compared LoRA with full fine-tuning behavior. In our first re-implementation, LoRA maintained more stable validation loss across epochs, while full fine-tuning showed signs of overfitting as validation loss increased 3. In the second re-implementation, LoRA converged faster than full fine-tuning, reaching strong validation accuracy early in training 4. Overall, these results indicate that LoRA can adapt RoBERTa efficiently while using only a small number of trainable parameters.

5. Discussion

The rank sweep suggests that LoRA does not require a large rank to perform well on these classification tasks. For SST-2, increasing the rank improved accuracy at first, but the curve flattened after rank 8, with little additional benefit from ranks 16 or 32 1. QNLI showed a similar pattern, although its accuracy continued to increase slightly at larger ranks 2. This supports the idea that the useful fine-tuning update for these tasks can be represented in a relatively low-dimensional subspace.

The biggest limitations came from the lack of computational power. Our experiments were trained on a single T4 GPU in Google Colab for 3 to 5 epochs, with each epoch taking approximately 5 minutes. Although the A100 is an upgrade over the V100 used in the paper [2], the paper trained on LoRA for 60 epochs on the SST-2 dataset and for 25 epochs on the QNLI dataset. Thus, we were unable to match the paper's experiments. However, our results show that even with fewer epochs, we obtained comparable results.

Our results also show that implementation details matter. Changes in learning rate, dropout, batch size, number of epochs, and whether LoRA layers are manually inserted or injected through PEFT can affect training behavior. This was especially visible when comparing our two re-implementations. The later PEFT-based version was easier to apply consistently across ranks and datasets, making it more suitable for systematic experiments 4.

6. Conclusion

We re-implemented LoRA on RoBERTa-base and evaluated it on SST-2 and QNLI. Across both tasks, LoRA significantly outperformed the unfine-tuned baseline while training only a small set of low-rank adapter parameters and the classification head. The rank experiments showed that performance improves quickly at small ranks and then begins to plateau, especially on SST-2. This supports the central claim of the LoRA paper that low-rank updates are often sufficient for adapting pretrained language models to downstream tasks.

Overall, our experiments show that LoRA is a practical and efficient alternative to full fine-tuning for classification tasks. It reduces the number of trainable parameters, avoids storing a separate full model for each task, and can still achieve strong validation accuracy. Future work could include running a more carefully tuned full fine-tuning baseline, measuring GPU memory usage directly, and testing additional GLUE tasks to better understand when larger LoRA ranks are useful.

7. GitHub

Our GitHub repository can be accessed here: `https://github.com/spectre729/cs_4782_final`

References

[1] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685, 2021.

[2] Balaban, Michael. *"A100 vs V100 Deep Learning Benchmarks — Lambda."* Lambda.ai, Lambda, Inc., 28 Jan. 2021, `lambda.ai/blog/nvidia-a100-vs-v100-benchmarks`.

[3] Socher, Richard, et al. *"Recursive Deep Models for Semantic Compositionality over a Sentiment Treebank."* Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, 2013, pp. 1631–1642.

[4] Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). *SQuAD: 100,000+ questions for machine comprehension of text.* In Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (pp. 2383–2392)

[5] Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2018). *GLUE: A multi-task benchmark and analysis platform for natural language understanding*. Proceedings of the 2018 EMNLP Workshop BlackboxNLP.

[6] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). *RoBERTa: A robustly optimized BERT pretraining approach.* arXiv preprint arXiv:1907.11692.

[7] Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., Bossan, B., & Tietz, M. (2022). *PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods* `https://github.com/huggingface/peft`

A. Results

The following includes the original results from our experiments.

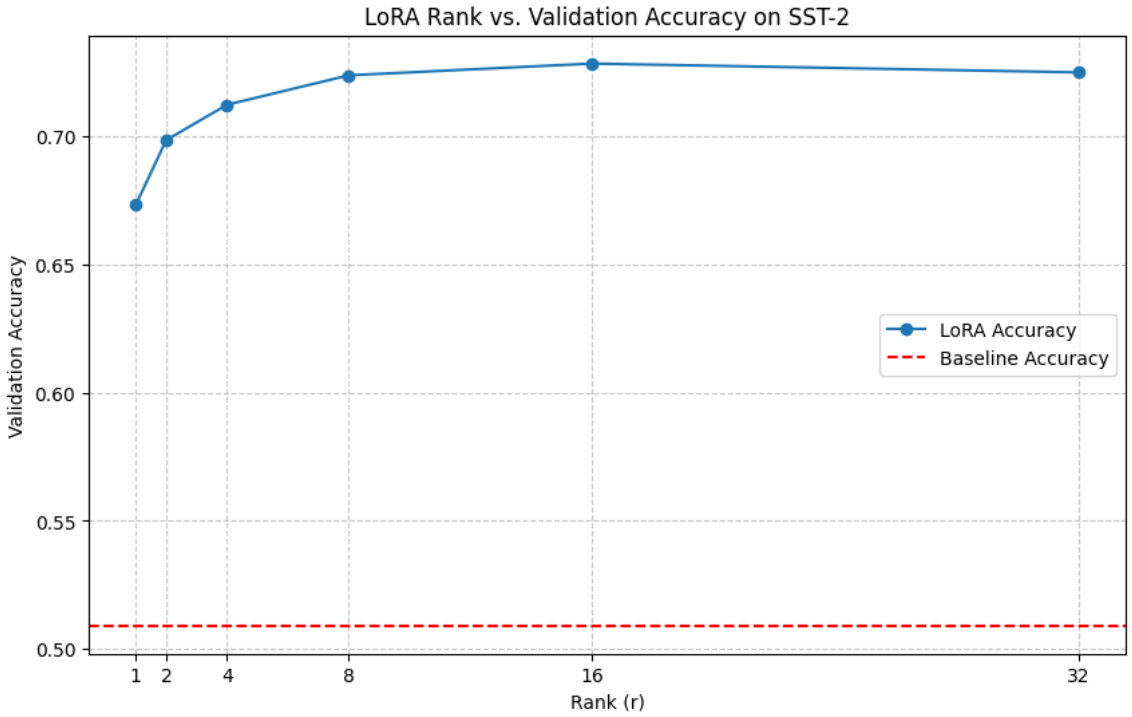

Figure 1: Experiment 1: LoRA's accuracy increases but stabilizes after rank = 8

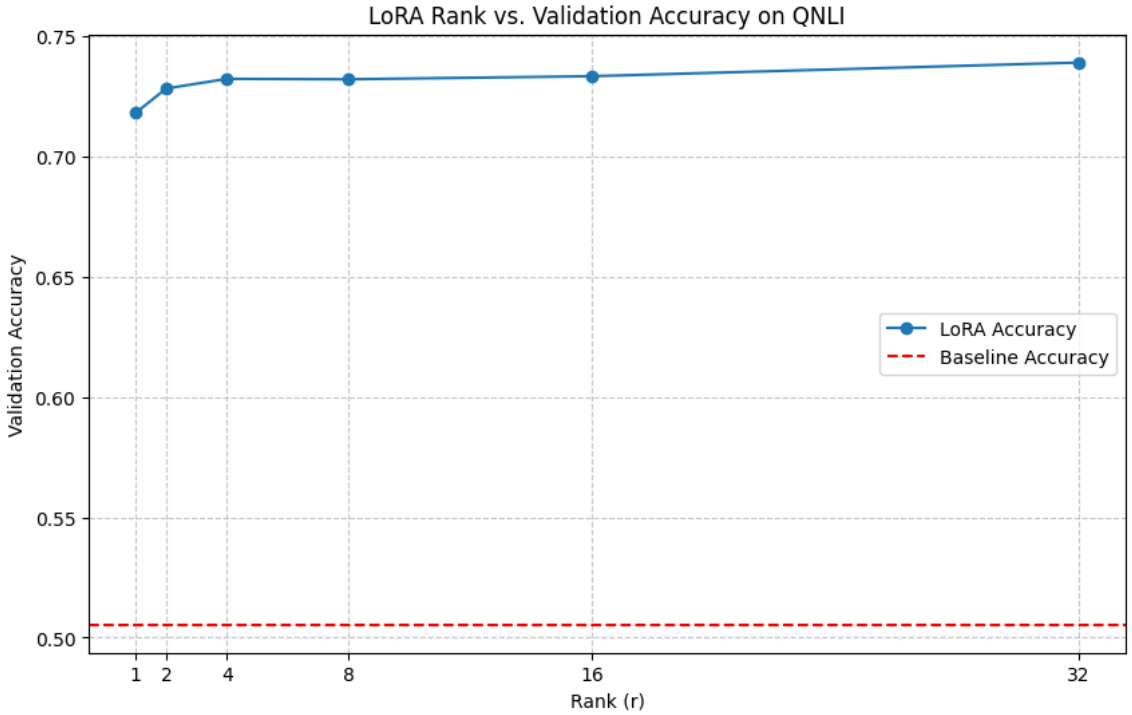

Figure 2: Experiment 2: LoRA's accuracy is generally around the same region regardless of rank

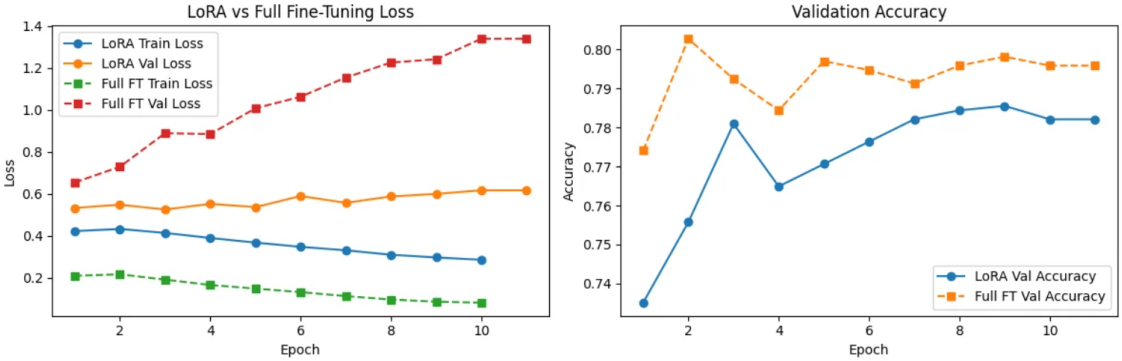

Figure 3: Re-Implementation 1: LoRA maintains stable validation loss, while full fine-tuning's validation loss rises steadily across epochs (overfitting)

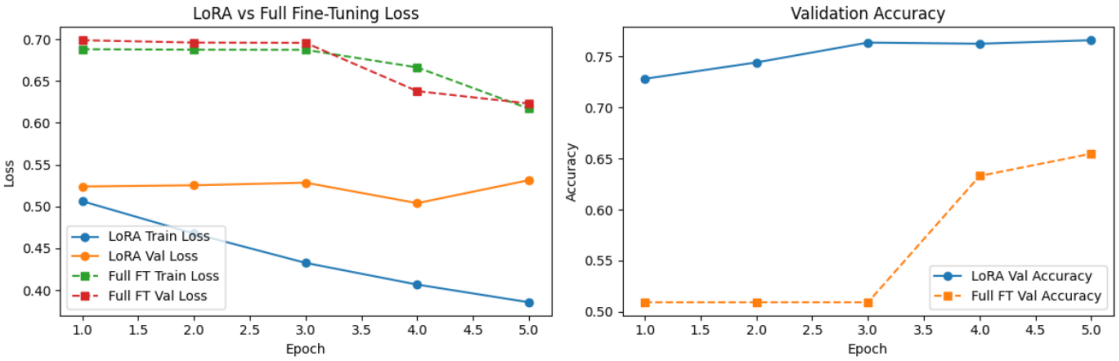

Figure 4: Re-Implementation 2: LoRA converges significantly faster than full fine-tuning, reaching 75% validation accuracy by epoch 1